THE NATIONAL COLLEGE

(AUTONOMOUS)

BASAVANAGUDI, BENGALURU – 560004

MACHINE LEARNING LAB REPORT

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF UNDER GRADUATE DEGREE

BACHELOR OF COMPUTER APPLICATION

BY

NAME(UUCMS.NO)

DEPARTMENT OF COMPUTER SCIENCE

YEAR 2023-2024

THE NATIONAL COLLEGE

(AUTONOMOUS)

BASAVANAGUDI, BENGALURU – 560004

॥ श्रद्धाऽ परमा गतिः ॥

Certificate

This is to certify that the "Machine Learning Laboratory Report" is a bonafide work carried out by _____ in the partial fulfilment for the award of Sixth Semester Bachelor of Computer Applications of The National College, Basavanagudi, Bengaluru – 560004, during the year 2023-2024.

_____

Teacher In-Charge

Examiner's Signature

1._____

2._____

_____

Head of Department

Prof. Ravi Hegde

Date:_____

INDEX

1.Write a program to Load and explore the dataset of. CVS and excel files using pandas.

Steps:

- **Install Pandas**: If Pandas is not installed, use `!pip install pandas` to install it. This command is typically run in a Jupyter notebook or a similar environment.
- **Import Pandas**: Import the Pandas library to use its functionalities.
- **Load CSV File**: Use `pd.read_csv()` to read the CSV file into a DataFrame. Replace `'path/to/your/csvfile.csv'` with the actual path to your CSV file.
- **Load Excel File**: Use `pd.read_excel()` to read the Excel file into a DataFrame. Replace `'path/to/your/excelfile.xlsx'` with the actual path to your Excel file. You can also specify the sheet name if the Excel file contains multiple sheets.
- **Explore Data**: Use methods like `.head()` to display the first few rows, `.info()` to get a concise summary of the DataFrame, and `.describe()` to generate descriptive statistics.

Program:

Loading Data from CSV

```
import pandas as pd
df=pd.read_csv("IRIS.csv")
print(df)
df.head()
df.info()
df.describe()
```

Loading Data from the sklearn Library

```
from sklearn.datasets import load_iris
iris_dataset = load_iris()
print(type(iris_dataset))
print(iris_dataset.items())
print("Target names:
{}".format(iris_dataset['target_names']))
print("Feature names:
\n{}".format(iris_dataset['feature_names']))
```

```
print("Type of data:
{}".format(type(iris_dataset['data'])))
print("Shape of data:
{}".format(iris_dataset['data'].shape))
print("First five columns of
data:\n{}".format(iris_dataset['data'][:5]))
```

Loading Data from EXCEL

```
import pandas as pd
exc=pd.ExcelFile(r"The_Excel_File_Path")
e=pd.read_excel(exc,sheet_name=0)
print(e)
```

Output:

Loading Data from CSV

```
     sepal_length  sepal_width  petal_length  petal_width         species
0             5.1          3.5           1.4          0.2     Iris-setosa
1             4.9          3.0           1.4          0.2     Iris-setosa
2             4.7          3.2           1.3          0.2     Iris-setosa
3             4.6          3.1           1.5          0.2     Iris-setosa
4             5.0          3.6           1.4          0.2     Iris-setosa
..            ...          ...           ...          ...             ...
145           6.7          3.0           5.2          2.3  Iris-virginica
146           6.3          2.5           5.0          1.9  Iris-virginica
147           6.5          3.0           5.2          2.0  Iris-virginica
148           6.2          3.4           5.4          2.3  Iris-virginica
149           5.9          3.0           5.1          1.8  Iris-virginica
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   sepal_length  150 non-null    float64
 1   sepal_width   150 non-null    float64
 2   petal_length  150 non-null    float64
 3   petal_width   150 non-null    float64
 4   species       150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

	sepal_length	sepal_width	petal_length	petal_width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.054000	3.758667	1.198667
std	0.828066	0.433594	1.764420	0.763161
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

Loading Data from the sklearn Library

```
<class 'sklearn.utils._bunch.Bunch'>
dict_items([('data', array([[5.1, 3.5, 1.4, 0.2],
       [4.9, 3. , 1.4, 0.2],
       [4.7, 3.2, 1.3, 0.2],
       [4.6, 3.1, 1.5, 0.2],
       [5. , 3.6, 1.4, 0.2]]))
Target names: ['setosa' 'versicolor' 'virginica']
Feature names:
['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width
(cm)']
Type of data: <class 'numpy.ndarray'>
Shape of data: (150, 4)
First five columns of data:
[[5.1 3.5 1.4 0.2]
 [4.9 3.  1.4 0.2]
 [4.7 3.2 1.3 0.2]
 [4.6 3.1 1.5 0.2]
 [5.  3.6 1.4 0.2]]
```

Loading Data from EXCEL

```
   Reg no Name  Avg
0       1    A   96
1       2    B   94
2       3    C   85
3       4    D   77
4       5    E   91
```

2.Write a program to visualize the dataset to gain insights using Matplotlib or Seaborn by plotting scatter plots, bar charts.

Steps:

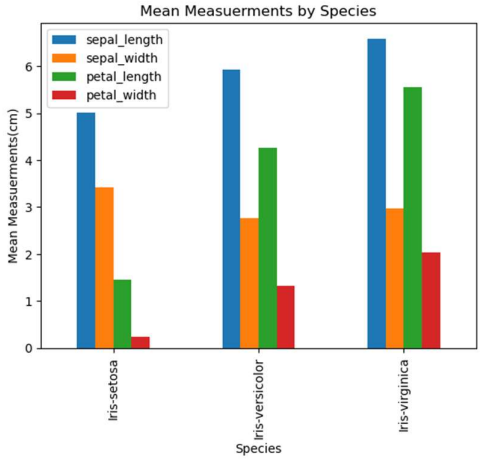
- **Install Required Libraries**: Install Pandas, Matplotlib, and Seaborn if they are not already installed using `!pip install pandas matplotlib seaborn`.
- **Import Libraries**: Import Pandas for data manipulation, Matplotlib for plotting, and Seaborn for enhanced visualization.
- **Load Data**: Load your dataset using Pandas. Adjust the file path and format (CSV or Excel) as needed.
- **Visualize Data**:
  - **Scatter Plot**: Use `sns.scatterplot()` to create a scatter plot. Replace `'column_x'` and `'column_y'` with the actual column names from your dataset.
  - **Bar Chart**: Use `sns.barplot()` to create a bar chart. Replace `'category_column'` and `'value_column'` with the actual column names from your dataset.
- **Show Plots**: Use `plt.show()` to display the plots.

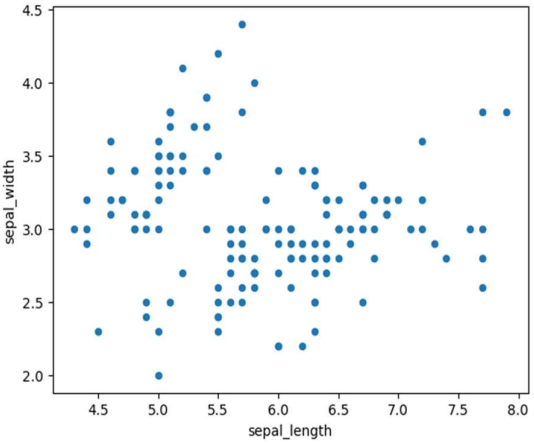
Program:

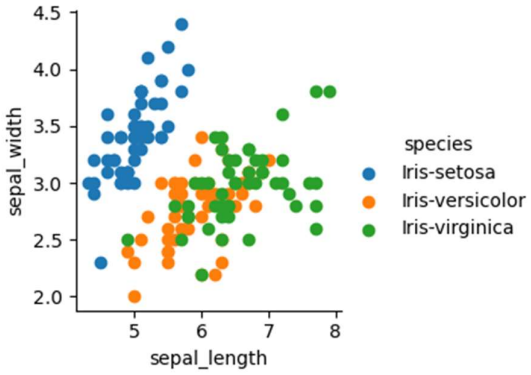
```
import pandas as pd
import matplotlib.pyplot as plt

df=pd.read_csv(R"D:\amara\IRIS.csv")

mean_measuerments=df.groupby('species').mean()
mean_measuerments.plot(kind='bar')
plt.xlabel('Species')
plt.ylabel('Mean Measuerments(cm)')
plt.title('Mean Measuerments by Species')
plt.show()

import seaborn as sns
df.plot(kind="scatter",x="sepal_length",y="sepal_widt
h")
plt.show()
```

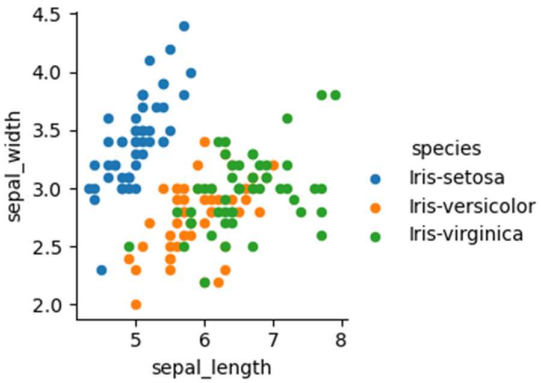
```
import seaborn as sns
sns.FacetGrid(df,hue="species").map(plt.scatter,"sepa
l_length","sepal_width") \
.add_legend()
plt.show()
```

```
import seaborn as sns
sns.FacetGrid(df,hue="species").map(plt.scatter,"sepa
l_length","sepal_width",s=20) \
.add_legend()
plt.show()
```









3.Write a program to visualize the dataset to gain insights using Matplotlib or Seaborn by plotting Histogram and Box plot using excel file.

Steps:

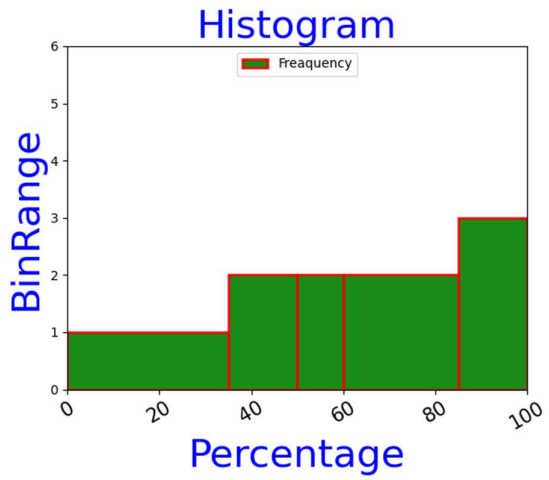
- **Install Required Libraries**: Install Pandas, Matplotlib, and Seaborn if they are not already installed using `!pip install pandas matplotlib seaborn openpyxl`.
- **Import Libraries**: Import Pandas for data manipulation, Matplotlib for plotting, and Seaborn for enhanced visualization.
- **Load Data**: Load your Excel dataset using Pandas. Adjust the file path and sheet name as needed.
- **Visualize Data**:
    - **Histogram**: Use `sns.histplot()` to create a histogram. Replace `'numerical_column'` with the actual numerical column name from your dataset. The `bins` parameter controls the number of bins, and `kde=True` adds a kernel density estimate.
    - **Box Plot**: Use `sns.boxplot()` to create a box plot. Replace `'numerical_column'` with the actual numerical column name from your dataset.
    - **Box Plot for a Categorical Variable**: Use `sns.boxplot()` to create a box plot comparing a numerical column against a categorical column. Replace `'category_column'` and `'numerical_column'` with the actual column names from your dataset.
- **Show Plots**: Use `plt.show()` to display the plots.

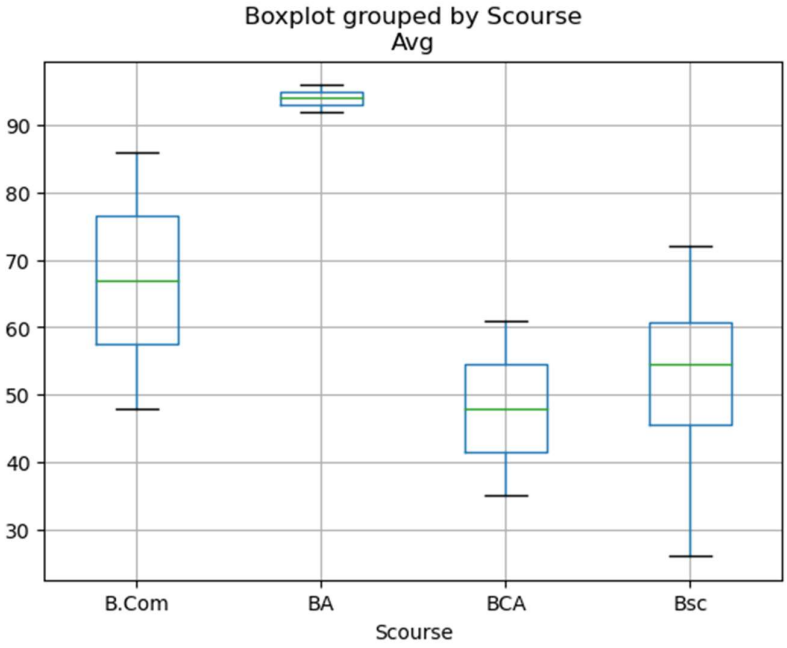
Program:

```
import matplotlib.pyplot as plt
import pandas as pd
exc=pd.ExcelFile(r"D:\amara\ml lab.xlsx")
s1=pd.read_excel(exc,sheet_name=0)

bin_ed=[0,35,50,60,85,100]
plt.hist(s1.Avg,bins=bin_ed,color='g',label='Freaquen
cy',alpha=0.9,edgecolor='r',orientation='vertical',li
newidth=2)
plt.legend(loc=9)
```

```
plt.xlabel('Percentage',color='b',size=30)
plt.ylabel('BinRange',color='b',size=30)
plt.xticks(rotation=30,fontsize=15)
plt.margins(x=0,y=1)
plt.title("Histogram",color='b',size=30)
s1.boxplot(column='Avg',by='Scourse')
plt.show()
```

Output:





4.Write a program to implement confusion matrix.

Steps:

- **Install Required Libraries**: Install Pandas, Scikit-Learn, Matplotlib, and Seaborn if they are not already installed using `!pip install pandas scikit-learn matplotlib seaborn`.
- **Import Libraries**: Import Pandas for data manipulation, Scikit-Learn for machine learning, and Matplotlib and Seaborn for plotting.
- **Load Data**: Load your dataset using Pandas. Adjust the file path and format (CSV or Excel) as needed.
- **Preprocess Data**:
    - Handle missing data using `SimpleImputer`.
    - Encode categorical variables using `LabelEncoder`.
    - Scale numerical features using `StandardScaler`.
- **Split Data**: Use `train_test_split` to split the data into training and testing sets. `test_size=0.2` means 20% of the data is used for testing.
- **Train a Classifier**: Initialize and train a classifier (e.g., Decision Tree) using the training data.
- **Predict and Compute Confusion Matrix**: Use the trained model to predict the target values for the test set and compute the confusion matrix using `confusion_matrix`.
- **Visualize Confusion Matrix**: Use Seaborn's `heatmap` to visualize the confu sion matrix. Customize the plot with titles and labels.

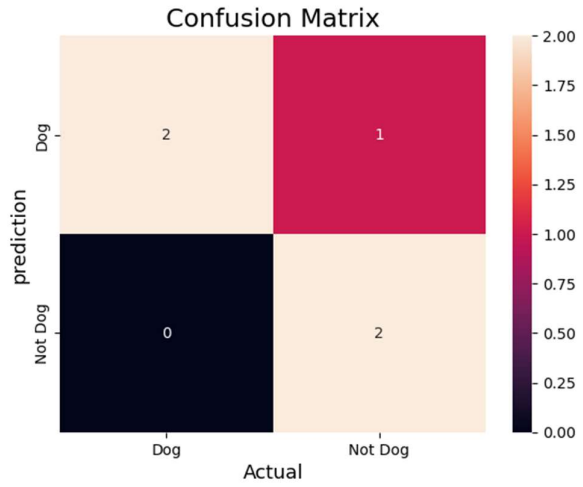
Program:

```
import numpy as np
from sklearn.metrics import
confusion_matrix,classification_report
import seaborn as sns
import matplotlib.pyplot as plt
actual=np.array(['Dog','Dog','Dog','Not Dog','Not
Dog',])
predicted=np.array(['Dog','Dog','Not Dog','Not
Dog','Not Dog'])
cm=confusion_matrix(actual,predicted)
sns.heatmap(cm,
            annot=True,
```

```
            fmt='g',
            xticklabels=['Dog','Not Dog'],
            yticklabels=['Dog','Not Dog'])
plt.ylabel('prediction',fontsize=13)
plt.xlabel('Actual',fontsize=13)
plt.title('Confusion Matrix',fontsize=17)
plt.show()
print(classification_report(actual,predicted))
```

Output:

```
              precision    recall  f1-score   support

         Dog       1.00      0.67      0.80         3
     Not Dog       0.67      1.00      0.80         2

    accuracy                           0.80         5
   macro avg       0.83      0.83      0.80         5
weighted avg       0.87      0.80      0.80         5
```



5.Write a program to Handle missing data, encode categorical variables, and perform feature scaling.

Steps:

- **Install Required Libraries**: Install Pandas and Scikit-Learn if they are not already installed using `!pip install pandas scikit-learn`.
- **Import Libraries**: Import Pandas for data manipulation and Scikit-Learn for preprocessing.
- **Load Data**: Load your dataset using Pandas. Adjust the file path and format (CSV or Excel) as needed.
- **Handle Missing Data**:
  - Use `SimpleImputer` to fill missing values. In this example, missing numerical values are filled with the mean value of the column.
  - Alternatively, drop rows with missing values using `dropna()`.
- **Encode Categorical Variables**:
  - **One-Hot Encoding**: Use `pd.get_dummies()` to convert categorical variables into a one-hot encoded format.
  - **Label Encoding**: Use `LabelEncoder` to convert categorical variables into numerical labels.
- **Feature Scaling**:
  - **Standard Scaling**: Use `StandardScaler` to scale features so that they have a mean of 0 and a standard deviation of 1.
  - **Min-Max Scaling**: Use `MinMaxScaler` to scale features to a range between 0 and 1.

Program:

```
import pandas as pd
from sklearn.impute import SimpleImputer

df=pd.read_csv(r"D:\amara\ML lab DataSets\car.csv")

print("Original DataFrame:")
print(df)
```

```
print("\nMissing values in each column:")
print(df.isnull().sum())

num_imputer=SimpleImputer(strategy='mean')
df[['year', 'price', 'mileage']] =
num_imputer.fit_transform \
(df[['year', 'price', 'mileage']])

cat_imputer=SimpleImputer(strategy='most_frequent')
df[['make', 'model']] =
cat_imputer.fit_transform(df[['make', 'model']])

print("\nDataFrame after handling missing values:")
print(df)
```

Output:

```
Original DataFrame:
        make       model    year    price  mileage
0     Toyota     Corolla  2010.0   7000.0  75000.0
1       Ford        F150  2015.0      NaN  50000.0
2      Honda       Civic  2018.0  15000.0      NaN
3     Toyota       Camry  2012.0   9000.0  60000.0
4        NaN         NaN  2016.0  10000.0  40000.0
5  Chevrolet      Malibu  2014.0  12000.0  30000.0
6       Ford       Focus     NaN   8500.0  45000.0
7     Nissan      Altima  2013.0   9500.0  55000.0
8      Honda      Accord  2017.0      NaN  70000.0
9     Toyota  Highlander  2019.0  25000.0  15000.0

Missing values in each column:
make       1
model      1
year       1
price      2
mileage    1
dtype: int64

DataFrame after handling missing values:
        make       model         year    price       mileage
0     Toyota     Corolla  2010.000000   7000.0  75000.000000
1       Ford        F150  2015.000000  12000.0  50000.000000
2      Honda       Civic  2018.000000  15000.0  48888.888889
3     Toyota       Camry  2012.000000   9000.0  60000.000000
4     Toyota      Accord  2016.000000  10000.0  40000.000000
5  Chevrolet      Malibu  2014.000000  12000.0  30000.000000
6       Ford       Focus  2014.888889   8500.0  45000.000000
7     Nissan      Altima  2013.000000   9500.0  55000.000000
8      Honda      Accord  2017.000000  12000.0  70000.000000
9     Toyota  Highlander  2019.000000  25000.0  15000.000000
```

6.Write a program to implement a decision tree classifier using sciki t-learn and visualize the decision tree and understand its splits.

Steps:

- **Install Required Libraries**: Install Pandas, Scikit-Learn, and Matplotlib if they are not already installed using `!pip install pandas scikit-learn matplotlib`.
- **Import Libraries**: Import Pandas for data manipulation, Scikit-Learn for machine learning, and Matplotlib for plotting.
- **Load Data**: Load your dataset using Pandas. Adjust the file path and format (CSV or Excel) as needed.
- **Preprocess Data**:
  - Handle missing data using `SimpleImputer`.
  - Encode categorical variables using `LabelEncoder`.
  - Scale numerical features using `StandardScaler`.
- **Split Data**: Use `train_test_split` to split the data into training and testing sets. `test_size=0.2` means 20% of the data is used for testing.
- **Train Decision Tree Classifier**: Initialize and train the decision tree classifier using the training data.
- **Visualize the Decision Tree**: Use `plot_tree` to visualize the decision tree. Customize the visualization with parameters like `filled`, `feature_names`, and `class_names`.
- **Evaluate Performance**:
  - Use `accuracy_score` to compute the accuracy of the classifier.
  - Use `confusion_matrix` to generate the confusion matrix.
  - Use `classification_report` to generate a detailed classification report including precision, recall, and F1-score.

Program:

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn import metrics
play1=pd.read_csv(r"D:\amara\ML lab
DataSets\PlayTennis.csv")
print(play1)
from sklearn.preprocessing import LabelEncoder
```

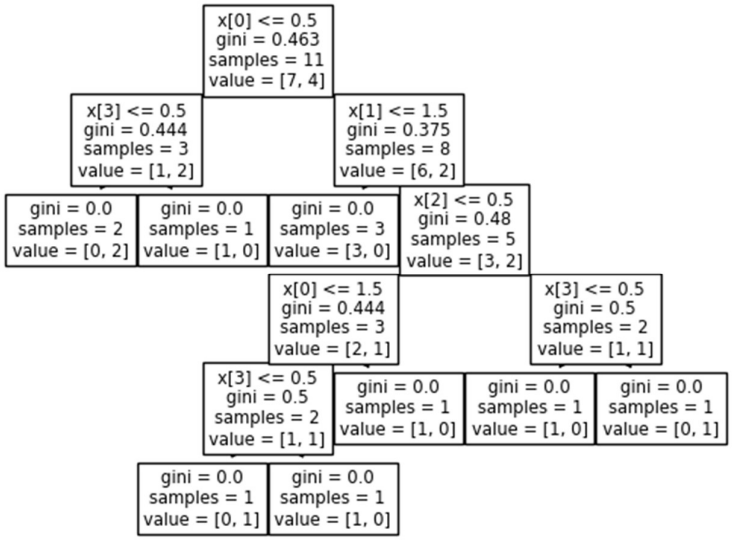
```
outlook=LabelEncoder()
temp=LabelEncoder()
humidity=LabelEncoder()
windy=LabelEncoder()
play=LabelEncoder()
Le=LabelEncoder()
play1['outlook']=Le.fit_transform(play1['outlook'])
play1['temp']=Le.fit_transform(play1['temp'])
play1['humidity']=Le.fit_transform(play1['humidity'])
play1['windy']=Le.fit_transform(play1['windy'])
play1['play']=Le.fit_transform(play1['play'])
features_cols=['outlook','temp','humidity','windy']
x=play1[features_cols]
y=play1.play
x_train,x_test,y_train,y_test=train_test_split(x,y,te
st_size=0.2,random_state=0)
print(x_train)
print(x_test)
print(y_train)
print(y_test)
from sklearn.tree import plot_tree
from sklearn.tree import export_text
clf=DecisionTreeClassifier(criterion="gini")
clf=clf.fit(x_train,y_train)
print(clf)
plot_tree(clf)
```

Output:

```
   outlook  temp humidity  windy play
0      sunny   hot     high  False   no
1      sunny   hot     high   True   no
2   overcast   hot     high  False  yes
3      rainy  mild     high  False  yes
4      rainy  cool   normal  False  yes
5      rainy  cool   normal   True   no
6   overcast  cool   normal   True  yes
7      sunny  mild     high  False   no
8      sunny  cool   normal  False  yes
9      rainy  mild   normal  False   no
10     sunny  mild   normal   True  yes
11  overcast  mild     high   True   no
12  overcast   hot   normal  False  yes
13     rainy  mild     high   True   no
```

```
  outlook  temp  humidity  windy
11        0     2         0      1
2         0     1         0      0
13        1     2         0      1
9         1     2         1      0
1         2     1         0      1
7         2     2         0      0
10        2     2         1      1
3         1     2         0      0
0         2     1         0      0
5         1     0         1      1
12        0     1         1      0
   outlook  temp  humidity  windy
8        2     0         1      0
6        0     0         1      1
4        1     0         1      0
11    0
2     1
13    0
9     0
1     0
7     0
10    1
3     1
0     0
5     0
12    1
Name: play, dtype: int32
8    1
6    1
4    1
Name: play, dtype: int32

DecisionTreeClassifier()
[Text(0.36363636363636365, 0.9166666666666666, 'x[0] <= 0.5\ngini = 0.463\n
samples = 11\nvalue = [7, 4]'),
 Text(0.18181818181818182, 0.75, 'x[3] <= 0.5\ngini = 0.444\nsamples = 3\nv
alue = [1, 2]'),
 Text(0.09090909090909091, 0.5833333333333334, 'gini = 0.0\nsamples = 2\nva
lue = [0, 2]'),
 Text(0.2727272727272727, 0.5833333333333334, 'gini = 0.0\nsamples = 1\nval
ue = [1, 0]'),
 Text(0.5454545454545454, 0.75, 'x[1] <= 1.5\ngini = 0.375\nsamples = 8\nva
lue = [6, 2]'),
 Text(0.45454545454545453, 0.5833333333333334, 'gini = 0.0\nsamples = 3\nva
lue = [3, 0]'),
 Text(0.6363636363636364, 0.5833333333333334, 'x[2] <= 0.5\ngini = 0.48\nsa
mples = 5\nvalue = [3, 2]'),
 Text(0.45454545454545453, 0.4166666666666667, 'x[0] <= 1.5\ngini = 0.444\n
samples = 3\nvalue = [2, 1]'),
 Text(0.36363636363636365, 0.25, 'x[3] <= 0.5\ngini = 0.5\nsamples = 2\nval
ue = [1, 1]'),
 Text(0.2727272727272727, 0.08333333333333333, 'gini = 0.0\nsamples = 1\nva
lue = [0, 1]'),
 Text(0.45454545454545453, 0.08333333333333333, 'gini = 0.0\nsamples = 1\nv
alue = [1, 0]'),
 Text(0.5454545454545454, 0.25, 'gini = 0.0\nsamples = 1\nvalue = [1, 0]'),
```

```
 Text(0.8181818181818182, 0.4166666666666667, 'x[3] <= 0.5\ngini = 0.5\nsam
ples = 2\nvalue = [1, 1]'),
 Text(0.7272727272727273, 0.25, 'gini = 0.0\nsamples = 1\nvalue = [1, 0]'),
 Text(0.9090909090909091, 0.25, 'gini = 0.0\nsamples = 1\nvalue = [0, 1]')]
```



7.Write a program to implement a k-Nearest Neighbours (k-NN) cla ssifier using scikit- learn and Train the classifier on the dataset and evaluate its performance.

Steps:

- **Install Required Libraries**: Install Pandas and Scikit-Learn if they are not already installed using `!pip install pandas scikit-learn`.
- **Import Libraries**: Import Pandas for data manipulation and Scikit-Learn for machine learning.
- **Load Data**: Load your dataset using Pandas. Adjust the file path and format (CSV or Excel) as needed.
- **Preprocess Data**:
  - Handle missing data using `SimpleImputer`.
  - Encode categorical variables using `LabelEncoder`.
  - Scale numerical features using `StandardScaler`.
- **Split Data**: Use `train_test_split` to split the data into training and testing sets. `test_size=0.2` means 20% of the data is used for testing.
- **Train k-NN Classifier**: Initialize and train the k-NN classifier using the training data. `n_neighbors=5` specifies the number of neighbors to use.
- **Evaluate Performance**:
  - Use `accuracy_score` to compute the accuracy of the classifier.
  - Use `confusion_matrix` to generate the confusion matrix.
  - Use `classification_report` to generate a detailed classification report including precision, recall, and F1-score.

Program:

```
import pandas as pd
import numpy as np
from sklearn import metrics
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix
print("Setup Complete")
df=pd.read_csv(R"D:\amara\ML lab DataSets\IRIS.csv")
```

```
print(df.head())
print("Column names:",df.columns)
feature_columns=df.columns[:-1]
target_column=df.columns[-1]

X=df[feature_columns]
y=df[target_column]
X_train, X_test,
y_train,y_test=train_test_split(X,y,test_size=0.2,ran
dom_state=42)

#Set the Number Of Neighbors for KNN
k=3
knn=KNeighborsClassifier(n_neighbors=k)

#fit the model on the training data
knn.fit(X_train,y_train)
print(X_train.shape)
print(X_test.shape)
print(y_train.shape)
print(y_test.shape)
y_pred=knn.predict(X_test)

#calculate and print the acccuracy
accuracy=metrics.accuracy_score(y_test,y_pred)
print (f"Accuracy:{accuracy}")

#Complete the Confusion Matrix
conf_matrix=confusion_matrix(y_test,y_pred)
print(f"Confusion Matrix:\n{conf_matrix}")
knn.score(X_test,y_test)
y_pred=knn.predict(X_test)
from sklearn.metrics import classification_report
print (classification_report(y_test,y_pred))
import matplotlib.pyplot as plt
import seaborn as sn
plt.figure(figsize=(7,5))
sn.heatmap(conf_matrix,annot=True)
plt.xlabel("predicted")
plt.ylabel("Truth")
```

Output:

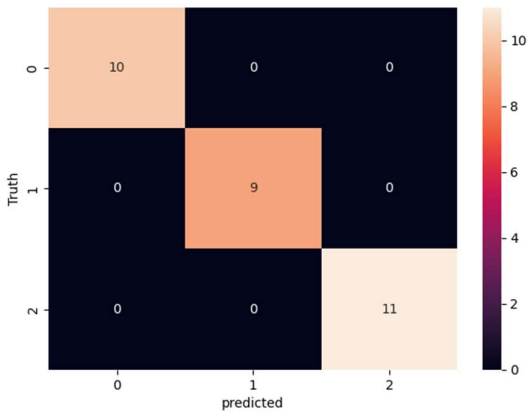
```
Setup Complete

   sepal_length  sepal_width  petal_length  petal_width      species
```

```
0           5.1          3.5           1.4          0.2  Iris-setosa
1           4.9          3.0           1.4          0.2  Iris-setosa
2           4.7          3.2           1.3          0.2  Iris-setosa
3           4.6          3.1           1.5          0.2  Iris-setosa
4           5.0          3.6           1.4          0.2  Iris-setosa

Column names: Index(['sepal_length', 'sepal_width', 'petal_length', 'petal_width',
       'species'],
      dtype='object')
```

KNeighborsClassifier

KNeighborsClassifier(n_neighbors=3)

```
(120, 4)
(30, 4)
(120,)
(30,)

Accuracy:1.0
Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]

1.0

                precision    recall  f1-score   support

    Iris-setosa       1.00      1.00      1.00        10
Iris-versicolor       1.00      1.00      1.00         9
 Iris-virginica       1.00      1.00      1.00        11

       accuracy                           1.00        30
      macro avg       1.00      1.00      1.00        30
   weighted avg       1.00      1.00      1.00        30

Text(58.222222222222214, 0.5, 'Truth')
```



8.Write a program to implement a linear regression model for regre ssion tasks and Train the model on a dataset with continuous target variables.

Steps:

- **Install Required Libraries**: Install Pandas and Scikit-Learn if they are not already installed using `!pip install pandas scikit-learn`.
- **Import Libraries**: Import Pandas for data manipulation and Scikit-Learn for machine learning.
- **Load Data**: Load your dataset using Pandas. Adjust the file path and format (CSV or Excel) as needed.
- **Preprocess Data**:
    - Handle missing data using `SimpleImputer`.
    - Encode categorical variables using `LabelEncoder`.
    - Scale numerical features using `StandardScaler`.
- **Split Data**: Use `train_test_split` to split the data into training and testing sets. `test_size=0.2` means 20% of the data is used for testing.
- **Train Linear Regression Model**: Initialize and train the linear regression model using the training data.
- **Evaluate Performance**:
    - Use `mean_squared_error` to compute the mean squared error of the predictions.
    - Use `r2_score` to compute the R-squared value, indicating the proportion of the variance in the dependent variable that is predictable from the independent variables.

Program:

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

# Load the CSV file into a DataFrame
dataset = pd.read_csv(r"D:\amara\ML lab
DataSets\experience.csv")
print(dataset)
```

```
# Split the dataset into features (X) and target (Y)
X = dataset.iloc[:, :-1].values
Y = dataset.iloc[:, 1].values


# Split the dataset into training and testing sets
X_Train, X_Test, Y_Train, Y_Test =
train_test_split(X, Y, test_size=1/3, random_state=0)
# Create and train the model
model = LinearRegression()
model.fit(X_Train, Y_Train)
# Get the model parameters
intercept, coefficients = model.intercept_,
model.coef_
print('Slope:', coefficients)
print('Y-intercept:', intercept)
# Make predictions
Y_Pred = model.predict(X_Test)
print(Y_Pred)
# Plot the training data and the regression line
plt.scatter(X_Train, Y_Train, color='red')
plt.plot(X_Train, model.predict(X_Train),
color='blue')
plt.title('Salary vs Experience (Training Set)')
plt.xlabel('Years of experience')
plt.ylabel('Salary')
plt.show()
# Plot the test data
plt.scatter(X_Test, Y_Test, color='red')
plt.plot(X_Train, model.predict(X_Train),
color='blue')
# Same line as training plot
plt.title('Salary vs Experience (Test Set)')
plt.xlabel('Years of experience')
plt.ylabel('Salary')
plt.show()
# Calculate and print the errors
Err = Y_Test - Y_Pred
print(Err)
# Plot the predicted values
plt.scatter(X_Test, Y_Pred, color='red')
plt.title('Salary vs Experience (Predicted Values)')
plt.xlabel('Years of experience')
plt.ylabel('Salary')
```

```
plt.show()
# Calculate and print the R^2 score
from sklearn.metrics import r2_score
r2 = r2_score(Y_Test, Y_Pred)
print('R^2 Score:', r2)
```

Output:

```
    YearsExperience    Salary
0               1.1   39343.0
1               1.3   46205.0
2               1.5   37731.0
3               2.0   43525.0
4               2.2   39891.0
5               2.9   56642.0
6               3.0   60150.0
7               3.2   54445.0
8               3.2   64445.0
9               3.7   57189.0
10              3.9   63218.0
11              4.0   55794.0
12              4.0   56957.0
13              4.1   57081.0
14              4.5   61111.0
15              4.9   67938.0
16              5.1   66029.0
17              5.3   83088.0
18              5.9   81363.0
19              6.0   93940.0
20              6.8   91738.0
21              7.1   98273.0
22              7.9  101302.0
23              8.2  113812.0
24              8.7  109431.0
25              9.0  105582.0
26              9.5  116969.0
27              9.6  112635.0
28             10.3  122391.0
29             10.5  121872.0

Slope: [9345.94244312]
Y-intercept: 26816.19224403119

[ 40835.10590871 123079.39940819  65134.55626083  63265.36777221
 115602.64545369 108125.8914992  116537.23969801  64199.96201652
  76349.68719258 100649.1375447 ]
```


```
[-3104.10590871  -688.39940819 -8053.55626083   -47.36777221
  1366.35454631  1305.1085008  -3902.23969801 -8405.96201652
  6738.31280742   652.8624553 ]
```


```
R^2 Score: 0.9749154407708353
```

9.Write a program to Implement K-Means clustering and Visualize c lusters.

Steps:

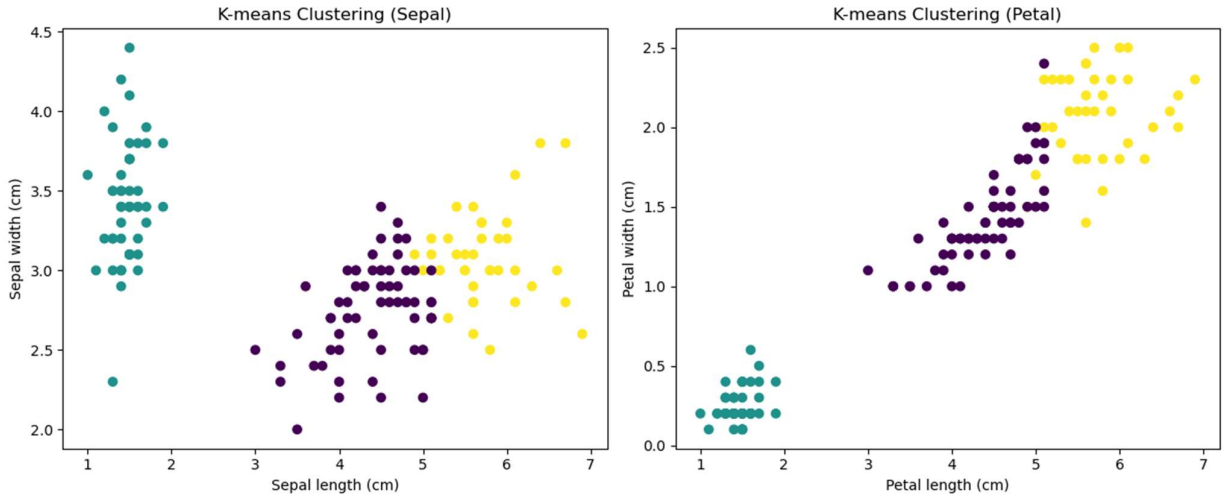
- **Install Required Libraries**: Install Pandas, Scikit-Learn, and Matplotlib if they are not already installed using `!pip install pandas scikit-learn matplotlib`.
- **Import Libraries**: Import Pandas for data manipulation, Scikit-Learn for machine learning, and Matplotlib for plotting.
- **Load Data**: Load your dataset using Pandas. Adjust the file path and format (CSV or Excel) as needed.
- **Preprocess Data**:
    - Handle missing data using `SimpleImputer`.
    - Encode categorical variables using `LabelEncoder`.
    - Scale numerical features using `StandardScaler`.
- **Train K-Means Model**: Initialize and train the K-Means model using the preprocessed data. Set `n_clusters` to the number of clusters you want to form.
- **Visualize Clusters**:
    - Use Seaborn's `scatterplot` to visualize the clusters. Set the `hue` parameter to the cluster labels and use `palette` to choose a color scheme.
    - Customize the plot with titles and labels.

Program:

```
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.cluster import KMeans

df=pd.read_csv(r"D:\amara\ML lab DataSets\IRIS.csv")

data=df.iloc[:,:-1]

kmeans=KMeans(n_clusters=3,random_state=0)
clusters=kmeans.fit_predict(data)

df["cluster"]=clusters

plt.figure(figsize=(12,5))
```

```
plt.subplot(1,2,1)
plt.scatter(data.iloc[:,2],data.iloc[:,1],
c=df['cluster'],cmap='viridis')
plt.xlabel('Sepal length (cm)')
plt.ylabel('Sepal width (cm)')
plt.title('K-means Clustering (Sepal)')

plt.subplot(1, 2, 2)
plt.scatter(data.iloc[:, 2], data.iloc[:, 3],
c=df['cluster'], cmap='viridis')
plt.xlabel('Petal length (cm)')
plt.ylabel('Petal width (cm)')
plt.title('K-means Clustering (Petal)')

plt.tight_layout()

plt.show()
```

Output:



10.Write a program to implement image segmentation using KMeans clustering algorithm.

Steps:

- **Install Required Libraries**: Install OpenCV, Scikit-Learn, Matplotlib, and NumPy if they are not already installed using `!pip install opencv-python-headless scikit-learn matplotlib numpy`.
- **Import Libraries**: Import OpenCV for image processing, Scikit-Learn for KMeans clustering, Matplotlib for plotting, and NumPy for numerical operations.
- **Load Image**: Load the image using OpenCV and convert it to RGB format.
- **Preprocess Image**: Reshape the image into a 2D array where each row is a pixel and each column is a color channel. Convert pixel values to float32 for clustering.
- **Apply KMeans Clustering**: Initialize and fit the KMeans model to the pixel data. Set the number of clusters `k` to the desired number of segments.
- **Post-process and Display Segmented Image**:
    - Map each pixel in the image to the color of its corresponding cluster center.
    - Reshape the clustered pixel labels back to the original image shape.
    - Display the original and segmented images side by side using Matplotlib.

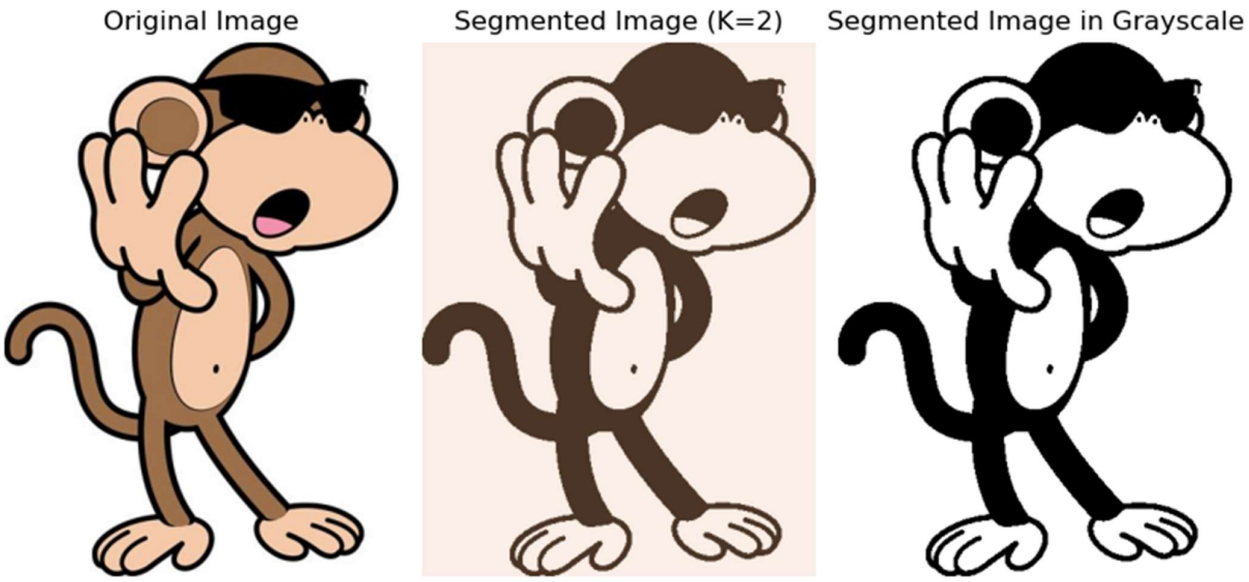
Program:

```
import numpy as np
import matplotlib.pyplot as plt
import cv2
from sklearn.cluster import KMeans

# Load the image using cv2.imread
image = cv2.imread("monkeyimg.jpg")

# Convert the image from BGR to RGB (cv2.imread reads
in BGR format)
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Reshape the image to a 2D array of pixels
(flattening it)
reshaped_image = image.reshape((-1, 3))
```

```
# Initialize the KMeans model
kmeans = KMeans(n_clusters=2, random_state=42)  # Specify the number of clusters (2 for binary segmentation)

# Fit the KMeans model to the reshaped image
kmeans.fit(reshaped_image)

# Get the cluster labels and cluster centers
cluster_labels = kmeans.labels_
cluster_centers = kmeans.cluster_centers_.astype('uint8')  # Convert cluster centers to uint8

# Create the segmented image by mapping each pixel to its cluster center
segmented_image = cluster_centers[cluster_labels]

# Reshape the segmented image to the original image shape
segmented_image = segmented_image.reshape(image.shape)

# Convert segmented image to grayscale
gray_segmented_image = cv2.cvtColor(segmented_image, cv2.COLOR_RGB2GRAY)

# Display the original image and the segmented image using matplotlib
plt.figure(figsize=(8, 5))

plt.subplot(1, 3, 1)
plt.imshow(image)
plt.title('Original Image')
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(segmented_image)
plt.title('Segmented Image (K=2)')
plt.axis('off')

plt.subplot(1, 3, 3)
plt.imshow(gray_segmented_image, cmap='gray')
plt.title('Segmented Image in Grayscale')
```

```
plt.axis('off')

plt.tight_layout()
plt.show()
```

Output:



11.Write a program to implement Naïve Bayes classifier algorithm.

Steps:

- **Load the Dataset**: Prepare the dataset for training and testing.
- **Calculate Prior Probabilities**: Compute the prior probabilities for each class.
- **Calculate Likelihood**: Compute the likelihood (conditional probability) for each feature given each class.
- **Calculate Posterior Probabilities**: Use Bayes' theorem to compute the posterior probabilities for each class given the input features.
- **Predict the Class**: Choose the class with the highest posterior probability.
- **Evaluate the Model**: Assess the performance of the model using a test dataset.

Program:

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.naive_bayes import GaussianNB
df=pd.read_csv(R"D:\amara\ML lab DataSets\weather.csv")
df
Numerics=LabelEncoder()
inputs=df.drop("Play",axis="columns")
target=df["Play"]
target
inputs['outlook_n']=Numerics.fit_transform(inputs['Outlook'])
inputs['Temp_n']=Numerics.fit_transform(inputs['Temp'])
inputs['Humidity_n']=Numerics.fit_transform(inputs['Humidity']
)
inputs['Windy_n']=Numerics.fit_transform(inputs['Windy'])
inputs
inputs_n=inputs.drop(["Outlook","Temp","Humidity","Windy"],axi
s="columns")
inputs_n
Classifier=GaussianNB()
Classifier.fit(inputs_n,target)
Classifier.score(inputs_n,target)
Classifier.predict([[0,0,0,1]])
accuracy=Classifier.score(inputs_n,target)
print(f"Accuracy of Naive Bayes Classifier: {accuracy:.2f}" )
prediction=Classifier.predict([[0,0,0,1]])
print(f"Prediction: {prediction}")
```

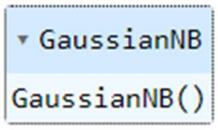
Output:

	Outlook	Temp	Humidity	Windy	Play
0	Rainy	Hot	High	f	no
1	Rainy	Hot	High	t	no
2	Overcast	Hot	High	f	yes
3	Sunny	Mild	High	f	yes
4	Sunny	Cool	Normal	f	yes
5	Sunny	Cool	Normal	t	no
6	Overcast	Cool	Normal	t	yes
7	Rainy	Mild	High	f	no
8	Rainy	Cool	Normal	f	yes
9	Sunny	Mild	Normal	f	yes
10	Rainy	Mild	Normal	t	yes
11	Overcast	Mild	High	t	yes
12	Overcast	Hot	Normal	f	yes
13	Sunny	Mild	High	t	no

```
0       no
1       no
2      yes
3      yes
4      yes
5       no
6      yes
7       no
8      yes
9      yes
10     yes
Name: Play, dtype: object
```

	Outlook	Temp	Humidity	Windy	outlook_n	Temp_n	Humidity_n	Windy_n
0	Rainy	Hot	High	f	1	1	0	0
1	Rainy	Hot	High	t	1	1	0	1
2	Overcast	Hot	High	f	0	1	0	0
3	Sunny	Mild	High	f	2	2	0	0
4	Sunny	Cool	Normal	f	2	0	1	0
5	Sunny	Cool	Normal	t	2	0	1	1
6	Overcast	Cool	Normal	t	0	0	1	1
7	Rainy	Mild	High	f	1	2	0	0
8	Rainy	Cool	Normal	f	1	0	1	0
9	Sunny	Mild	Normal	f	2	2	1	0
10	Rainy	Mild	Normal	t	1	2	1	1
11	Overcast	Mild	High	t	0	2	0	1
12	Overcast	Hot	Normal	f	0	1	1	0
13	Sunny	Mild	High	t	2	2	0	1

	outlook_n	Temp_n	Humidity_n	Windy_n
0	1	1	0	0
1	1	1	0	1
2	0	1	0	0
3	2	2	0	0
4	2	0	1	0
5	2	0	1	1
6	0	0	1	1
7	1	2	0	0
8	1	0	1	0
9	2	2	1	0
10	1	2	1	1
11	0	2	0	1
12	0	1	1	0
13	2	2	0	1




```
0.8571428571428571

array(['yes'], dtype='<U3')

Accuracy of Naive Bayes Classifier: 0.86

Prediction: ['yes']
```

12.Write a Program to implement approach for agglomerative clustering.

Steps:

- **Install Required Libraries**: Install Pandas, Scikit-Learn, Matplotlib, and SciPy if they are not already installed using `!pip install pandas scikit-learn matplotlib scipy`.
- **Import Libraries**: Import Pandas for data manipulation, Scikit-Learn for clustering, Matplotlib for plotting, and SciPy for hierarchical clustering dendrograms.
- **Load Data**: Load your dataset using Pandas. Adjust the file path and format (CSV or Excel) as needed.
- **Preprocess Data**:
  - Handle missing data using `SimpleImputer`.
  - Optionally, encode categorical variables using `LabelEncoder`.
  - Scale numerical features using `StandardScaler`.
- **Apply Agglomerative Clustering**:
  - Initialize the AgglomerativeClustering model with the desired number of clusters `n_clusters`.
  - Fit the model to the data and predict cluster labels.
- **Visualize Clusters**:
  - Optionally, plot a dendrogram to visualize hierarchical clustering.
  - Plot a scatter plot to visualize the clusters with different colors for each cluster.

Program:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import dendrogram,
linkage
from sklearn.cluster import AgglomerativeClustering

# Read the distance matrix from the CSV file
csv_file_path = 'distance_matrix.csv'
df = pd.read_csv(csv_file_path, index_col=0)
```

```
# Convert the DataFrame to a NumPy array
distance_matrix = df.values

# Perform agglomerative clustering using sklearn
agg_clustering = AgglomerativeClustering(
    affinity='precomputed',
    linkage='single',
    distance_threshold=0,
    n_clusters=None
)
agg_clustering.fit(distance_matrix)

# Create the linkage matrix
linked =
linkage(sch.distance.squareform(distance_matrix),
method='single')
# Plot the dendrogram
plt.figure(figsize=(7, 4))
dendrogram(linked, labels=df.index)
plt.title('Agglomerative Hierarchical Clustering
Dendrogram')
plt.xlabel('Cluster Label')
plt.ylabel('Distance')
plt.show()
```

Output: